FRACTL · STAGES 2–3 · MEMBER 2

Backend: Smaller Code, Faster Builds, a 3D Turtle

How the backend went from "one call per line drawn" to "one function per grammar rule", learned to compile in memory, and grew a 3D runtime — and why every old picture still comes out identical.

Pipeline: turtle IR / grammar → LLVM IR (**recursive** or unrolled) → [--opt] → **ORC JIT in memory** or .o + link → runtime.c (2D SVG or **3D .f3d stream**)

IN ONE BREATH

Stage 1 wrote one LLVM call for every single line of the drawing, so bigger pictures meant bigger, slower programs. Now the compiler writes **one LLVM function per grammar rule** that calls itself — the code is the size of the grammar, not the picture. LLVM's **JIT** turns it into machine code in memory and runs it straight away, and the runtime can draw in **3D**. The full meadow — 40,000 pieces — compiles and runs in about a tenth of a second.

01 What changed since Stage 1

Area	Stage 1	Now
Runtime	one continuous line; no move/push/pop	Stage 2 pen-up moves, a growable save/restore stack, one polyline per stroke Stage 3 a full 3D turtle writing a `.f3d` segment stream; a seeded random-number function
Code generation	unrolled: one call per command	Stage 3 **recursive**: one function per rule symbol (default); unrolled kept as `--unroll`
Running it	.o file + `gcc` link + run	Stage 3 also **LLVM ORC JIT** (`--jit`): no file, no linker
Optimizer	none	Stage 3 `--opt` : merges neighbouring turns
Scenes	—	Stage 3 instance table + loop + switch inside `fractl_main`
Driver flags	`-o --svg --emit-ir`	+ `--jit --unroll --opt --out --timings --ir-only`
Distribution	needs LLVM installed	one **fully static** `fractlc` with LLVM inside — runs on any WSL

02 Stage 2: the runtime learns to branch Stage 2

Stage 1's codegen already emitted `turtle_move` , `turtle_push` and `turtle_pop` — Stage 2 only had to make the runtime implement them.

- **Save/restore stack**: `turtle_push()` copies the turtle state onto a stack that doubles in size when full (tested 5,000 levels deep); `turtle_pop()` restores it. A pop with nothing saved stops with `state stack underflow` .
- **Strokes**: a pen-up move or a pop starts a new `<polyline>` . A push does *not* lift the pen, so a branch continues the line it starts from — the plant's 6,048 segments need only 2,048 polylines.

- **Stage 1 untouched**: same real-size SVG, same number format; triangle and Koch come out **byte-identical** to Stage 1 (tested).

03 Recursive code generation — the big idea Stage 3

The problem. Unrolled code grows with the picture: the Stage 2 plant was 21,063 calls in a single function, a 1 MB object file. A meadow with a tree and 1,300 tufts would be 200,000 lines of LLVM IR.

The idea. An L-system is naturally recursive: "X becomes F+[[X]-X]…" means *draw X by drawing F, turning, then drawing X again, one level deeper*. So each rule symbol becomes an LLVM function that takes `d` = how many rewrite passes are left:

```
define internal void @Plant.X(i32 %d) {
entry:
  %done = icmp eq i32 %d, 0              ; no passes left?
  br i1 %done, label %leaf, label %grow
leaf:                                     ; X draws nothing at the last level
  ret void
grow:
  %next = sub i32 %d, 1
  call void @Plant.F(i32 %next)           ; X -> "F+[[X]-X]-F[-FX]+X"
  call void @turtle_turn(double 2.500000e+01)
  call void @turtle_push()
  call void @turtle_push()
  call void @Plant.X(i32 %next)           ; recursion, one level deeper
  call void @turtle_pop()
  …
```

The axiom becomes a small function that calls the rule functions with `d = iterations`. Symbols with no rule are drawn on the spot (the shared `terminalCmd()` from Member 1 says what each symbol draws).

Random rules in LLVM IR

A symbol with several weighted productions asks the runtime for a number and branches — exactly the arithmetic Member 1's rewriter does at compile time:

```
grow:
  %level = sub i32 2, %d                  ; which rewrite pass this is
  %r = call double @fractl_rand(i32 %level)
  %x = fmul double %r, 1.000000e+00       ; × total weight
  %0 = fcmp olt double %x, 3.000000e-01   ; first production's share
  br i1 %0, label %p0, label %t1          ; p0 = production 0, t1 = test the next
```

Program	Unrolled IR	Recursive IR	Object file	Object emit time
Stage 2 plant	21,086 lines	**70 lines**	1.0 MB → **2.7 KB**	0.88 s → 0.08 s
Dragon	12,309 lines	**60 lines**	751 KB → **2.5 KB**	0.44 s → 0.17 s
Meadow (1,301 instances)	201,463 lines	**531 lines**	11.9 MB → **91 KB**	4.58 s → 0.17 s

> SAY IT LIKE THIS
> "Stage 2 compiled the picture. Stage 3 compiles the grammar. The plant went from 21,000 lines of IR to 70, and the drawing is byte-for-byte the same."

04 Scenes: a table and a loop Stage 3

Placing 1,301 copies with 1,301 separate call sequences would undo the savings. Instead the instance list from Member 1's scene layout becomes a **constant table** in the module, and `fractl_main` is a real loop:

```
instance:                                         ; for i in 0 .. 1300
  %i   = phi i64 [ 0, %entry ], [ %i.next, %next ]
  %row = getelementptr [1301 x %instance], ptr @instances, i64 0, i64 %i
  …  load x, y, z, yaw, scale, seed, fractal  …
  switch i32 %fractal, label %next [ i32 0, label %grow.Tree
                                     i32 1, label %grow.Tuft ]
grow.Tuft:
  call void @turtle3_begin(… position, yaw, scale, width, taper …)
  call void @fractl_seed(i64 %seed)              ; this copy's own randomness
  call void @Tuft.axiom()
  call void @turtle3_end()
```

Uses LLVM **struct types, a global constant array, a phi node, GEP, loads and a switch** — real control flow, not just calls.

05 The `--opt` peephole pass Stage 3

`Backend::foldTurns()` walks every basic block and merges **directly adjacent** calls to the same rotation with constant angles: `turn(-60); turn(-60)` → `turn(-120)`. If a run adds up to a whole turn (a multiple of 360°) it is deleted completely.

- It never merges across a draw, a move or a push/pop — those change where the turn applies.
- It never merges **different axes** (yaw with pitch): 3D rotations don't commute.
- Every demo's drawing is tested to be **identical** with and without `--opt`.

06 ORC JIT: compile and run in memory Stage 3

`jit.cpp` uses LLVM's **LLJIT** (the ORC JIT framework). Instead of writing a `.o` and calling `gcc`:

1. Create an `LLJIT` for the host CPU, using the fast instruction selector (`-O0`) — our code is call sequences with constant arguments, so heavy optimization isn't worth the time.
2. Give it the runtime's functions as **absolute symbols**: `runtime.c` is linked into `fractlc` itself, and we tell the JIT the address of every `turtle_*` function.
3. `addIRModule()`, then `lookup("fractl_main")` — this is the moment LLVM generates machine code in memory.
4. Call the returned function pointer. The compiled code runs inside `fractlc` and writes the drawing.

Both phases are timed separately for the studio's pipeline panel (meadow: JIT ≈ 7 ms, run ≈ 55 ms).

> **SAME OUTPUT, EVERY ROUTE**
> Object-file + link and JIT run the very same runtime code, so their outputs are byte-identical — tested for all 17 demos and presets.

07 The 3D runtime Stage 3

The 3D turtle (`turtle3_*` in `runtime.c`) keeps a position, the three direction vectors **H, L, U**, a width and a colour index.

- **Rotations**: turn rotates H and L around U, pitch rotates H and U around L, roll rotates L and U around H. After each rotation the frame is re-orthonormalized (H normalized, L made perpendicular, U = H × L) so tiny floating-point errors never pile up.
- **Instances**: `turtle3_begin()` resets the turtle and records the copy's position, yaw and scale; every segment is converted to world coordinates on the way out.
- **Output — the `.f3d` stream**: one text line per piece, in exactly the order the program drew it. That order is what the studio uses to show the model being built.

```
o Tuft 12 #2f6b1f,#4f8f2d,#86b84a                    instance name, index, palette
t x0 y0 z0  x1 y1 z1  radius  colour                 tube (F): wood
b x0 y0 z0  x1 y1 z1  sx sy sz  width0 width1  colour   blade (G): grass / leaf, with its
sideways direction
```

- **Randomness**: `fractl_seed()` / `fractl_rand(level)` keep one counter per rewrite pass and use the same SplitMix64 formula as Member 1's `rng.hpp` — that is why recursive and unrolled builds choose identically.

08 The driver and the static build

```
fractlc <file.fr> --svg out.svg | --out out.f3d
       [-o out.o] [--jit] [--emit-ir out.ll] [--unroll] [--opt] [--timings] [--ir-only]
```

- `--timings` prints one JSON line with each phase's time — parse, IR generation, optimizer, JIT or object emission, run — which the studio displays live.
- `--ir-only` stops after code generation (the studio's "LLVM IR side by side" uses it).
- **Static build** (`make bundle`): `fractlc` is linked with `-static`, LLVM included, into one 35 MB file with no library dependencies. LLVM's support library mentions zstd compression, which FRACTL never uses, so a tiny stub file stands in for it. Result: the presentation laptop needs no LLVM at all.

09 Numbers to quote

70 lines of IR for the plant (was 21,086)	**531** lines of IR for the whole meadow (unrolled: 201,463)	**~7 ms** ORC JIT for the meadow
~55 ms running it: 39,588 tubes and blades written	**0.14 s** edit → compiled → run → on screen (round trip)	**0** changed pixels in any Stage 1–2 picture

10 Questions you may be asked

Why is recursive code allowed to replace the expanded string?

Rewriting all passes and then drawing gives the same sequence as drawing depth-first while recursing: each recursive call is "this symbol, expanded the remaining number of times". The tests compare both outputs byte for byte.

What does the JIT actually do that the old path didn't?

It skips the object file, the linker and the executable: LLVM produces machine code directly into memory inside `fractlc`, and we call it like a normal function.

Is the IR verified?

Yes — `verifyModule()` runs after code generation and again before object emission.

Why -O0 for the JIT?

The generated functions are short call sequences with constants; optimization would cost more time than it saves. The fast selector made the JIT step several times quicker with identical output.

Why can't `--opt` merge a pitch with a yaw?

Rotations about different axes don't commute: pitch-then-yaw points the turtle somewhere different from yaw-then-pitch.

How do you know the runtime's 3D maths is right?

The runtime unit tests check exact coordinates after turns, pitches, push/pop and instance rotation and scaling (36 runtime checks).

FRACTL Stages 2–3 · backend (Member 2) · codegen.hpp, codegen.cpp, emit_object.cpp, jit.cpp, runtime.c, runtime.h, fractlc.cpp, tools/zstd_stub.c